

Fachhochschule Südwestfalen
Fachbereich Informatik & Naturwissenschaften
Sommersemester 2026

Vertiefung Software Engineering

Thema

**Wartbare, erweiterbare und nachvollziehbare
Microservice-Architektur am Beispiel
eines verteilten Online-Shops**

Rahmenbedingungen	
Studiengang	Angewandte Informatik
Kurs	Vertiefung Software Engineering
Abgabedatum	Siehe Moodle-Kurs
Gruppengröße	2-3 Studierende pro Gruppe
Abgabeform	Moodle-Aufgabe (es gilt das Abgabedatum) Abschlussbericht (PDF) Präsentation (PDF) Quellcode
Bearbeitungsform	• Eigenständige Gruppenarbeit • Quellcode muss lauffähig sein

1 Motivation und Lernziele

Moderne E-Commerce-Plattformen stellen nahezu alle klassischen Qualitätsanforderungen an Software gleichzeitig: Sie müssen **hochverfügbar**, leicht **erweiterbar**, sicher, **auditierbar** und **performant** sein. Gleichzeitig kommunizieren mehrere unabhängige Services miteinander, und ein einziger fehlgeschlagener Schritt - etwa eine abgelehnte Zahlung oder ein leeres Lager - muss **konsistent** rückgängig gemacht werden.

Diese Aufgabe adressiert genau diesen Spannungsbogen. Die Gruppen entwerfen und implementieren eine verteilte Anwendungslandschaft, die folgende **nicht funktionale Anforderungen** in ihrer Architektur sichtbar macht:

- * **Wartbarkeit:** klare Schichtentrennung, definierte Schnittstellen, minimale Kopplung
- * **Erweiterbarkeit:** Fassaden- und Strategiemuster ermöglichen den Austausch von Zahlungsanbietern ohne Anpassungen im Kernsystem
- * **Nachvollziehbarkeit:** jeder Vorgang wird lückenlos protokolliert - strukturierte Logs und applikationsspezifische Audit-Snapshots in der Datenbank
- * **Resilienz:** schlägt ein Service-Aufruf fehl, wird der gesamte Vorgang konsistent über ein Saga-Muster zurückgerollt

2 Fachliche Beschreibung des Systems

Das zu entwickelnde System simuliert den Bestellprozess eines kleinen Online-Shops. Ein Kunde legt Artikel in den Warenkorb, schließt den Kauf ab und bezahlt. Das System prüft Lagerbestände, reserviert die Ware, initiiert die Zahlung, erzeugt eine Rechnung und schließt die Bestellung ab. Jeder dieser Schritte liegt in der Verantwortung eines **eigenen Microservices**.

2.1 Systemlandschaft (Pflicht)

Die folgende Tabelle beschreibt die Pflicht-Services, die jede Gruppe implementieren muss:

Service	Technologie (Vorschlag)	Verantwortlichkeit
Shop-Service	Spring Boot / Node.js	Produktkatalog, Warenkorb, Bestellauslösung. Einziger nach außen exponierter Service (API-Gateway-Rolle).
Billing-Service	Spring Boot / Node.js	Orchestriert den Zahlungsfluss. Kommuniziert ausschließlich über eine Payment-Fassade mit externen Anbietern.
Payment-Fassade	Modul im Billing-Service	Abstrahiert mind. zwei Zahlungsanbieter (z.B. Stripe-Stub, Klarna-Stub). Austauschbar per Konfiguration.
Invoice-Service	Spring Boot / Node.js	Erzeugt PDF-Rechnungen und speichert diese. Wird nach erfolgreicher Zahlung asynchron aufgerufen.
Warehouse-Service	Spring Boot / Node.js	Pseudo-Warenwirtschaft: prüft Lagerbestand, reserviert und bucht Ware aus. Kann gezielt Fehler werfen.
Audit-Service / Event-Store	Spring Boot / Node.js	Empfängt Domain-Events aller Services, persistiert unveränderliche Audit-Snapshots in der Datenbank.

2.2 Bestellprozess (Happy Path)

Der folgende Ablauf beschreibt einen erfolgreichen Bestellvorgang und muss vollständig implementiert werden:

- Schritt 1: **Bestellung einleiten:** Shop-Service empfängt POST /orders, validiert die Anfrage und weist eine eindeutige correlationId zu.
- Schritt 2: **Lager prüfen & reservieren:** Shop-Service ruft synchron den Warehouse-Service auf. Dieser prüft Bestände und legt eine Reservierung an.
- Schritt 3: **Zahlung initiieren:** Billing-Service wird aufgerufen. Er delegiert über die Payment-Fassade an den konfigurierten Anbieter.
- Schritt 4: **Zahlung bestätigen:** Zahlungsanbieter-Stub gibt Erfolg zurück. Billing-Service protokolliert das Ergebnis.
- Schritt 5: **Rechnungserstellung:** Billing-Service publiziert ein PaymentSucceeded-Event. Invoice-Service erstellt asynchron eine PDF-Rechnung.
- Schritt 6: **Lager ausbuchen:** Warehouse-Service bucht die reservierten Artikel endgültig aus.
- Schritt 7: **Bestellung abschließen:** Shop-Service setzt den Status auf COMPLETED und antwortet dem Client.

2.3 Fehlerszenario und Saga-Kompensation

Das System muss mit folgenden Fehlerszenarien umgehen und dabei alle bereits ausgeführten Schritte rückgängig machen (Kompensationstransaktionen gemäß dem Saga-Muster):

- * **Zahlung abgelehnt:** Reservierung im Warehouse-Service wird storniert. Bestellung erhält Status PAYMENT_FAILED.
- * **Lager nicht ausreichend:** Kein weiterer Aufruf erfolgt. Bestellung erhält Status OUT_OF_STOCK.
- * **Invoice-Service nicht erreichbar:** Zahlung wird nicht rückgängig gemacht. Stattdessen wird ein Retry-Mechanismus (mind. 3 Versuche) aktiviert und ein Fehlereintrag im Audit-Log angelegt.
- * **Warehouse-Ausbuchung schlägt fehl:** Zahlung wird rückerstattet (Refund über Fassade). Bestellung erhält Status ROLLBACK_COMPLETED.

Jeder Kompensationsschritt muss als eigener Audit-Snapshot in der Datenbank gespeichert werden.

3 Architekturanforderungen

3.1 Payment-Fassade (Pflicht)

Die Payment-Fassade ist das architektonische Herzstück dieser Aufgabe. Sie muss folgende Eigenschaften aufweisen:

- * **Einheitliche Schnittstelle:** alle Aufrufe von Billing-Service-Seite laufen ausschließlich über die Fassade; kein direkter Zugriff auf einen konkreten Anbieter.
- * **Mindestens zwei Anbieter:** z.B. StripeAdapter und KlarnaAdapter.
- * **Konfigurierbarkeit:** Der aktive Anbieter wird per Umgebungsvariable oder Konfigurationsdatei gesetzt, nicht per Code.
- * **Erweiterbarkeit:** Ein dritter Anbieter muss hinzufügbare sein, ohne bestehenden Code zu verändern (Open/Closed Principle).
- * **Operationen der Fassade:** `charge(orderId, amount, currency)`, `refund(transactionId, amount)`, `getStatus(transactionId)`.

3.2 Audit- und Snapshot-System (Pflicht)

Jeder Zustandsübergang eines Bestellvorgangs wird als unveränderlicher Datensatz in der Datenbank gespeichert (Event Sourcing Light). Ein Snapshot enthält mindestens:

Feld	Typ	Bedeutung
correlationId	UUID	Eindeutige ID des gesamten Bestellvorgangs; verbindet alle Snapshots einer Bestellung
eventType	String (Enum)	z.B. ORDER_PLACED, PAYMENT_INITIATED, PAYMENT_FAILED, ROLLBACK_COMPLETED
service	String	Herkunft des Events (shop-service, billing-service, ...)
timestamp	Timestamp	Exakter Zeitpunkt des Ereignisses (UTC)
payload	JSON	Zustandsabbild: relevante Felder des Domaenobjekts zum Zeitpunkt des Events
previousEventId	UUID (nullable)	Verkettung: Referenz auf den unmittelbar vorherigen Snapshot
actor	String	Systemkomponente oder Benutzer, die das Event ausgelöst hat
statusCode	String	SUCCESS, FAILURE, COMPENSATING, COMPENSATED

Anforderungen an das Audit-System:

- * Snapshots sind nach dem Einfügen unveränderlich (keine UPDATE- oder DELETE-Operationen auf der Tabelle).
- * Ein REST-Endpunkt `GET /audit/orders/{correlationId}` liefert alle Snapshots einer Bestellung chronologisch sortiert.
- * Der Audit-Service muss als eigener Microservice realisiert werden und darf kein Business-Wissen über Shop oder Zahlung besitzen.

3.3 Strukturiertes Logging (Pflicht)

Jeder Service führt ein strukturiertes Log (JSON-Format). Jeder Logeintrag enthält mindestens:

- * `correlationId` - verbindet Logzeilen eines Vorgangs über alle Services hinweg
- * `service` - Name des Services
- * `level` - INFO, WARN, ERROR, DEBUG
- * `timestamp` - ISO-8601 UTC
- * `message` - lesbarer Text
- * `context` - strukturiertes Objekt mit relevantem Zustand (z.B. orderId, amount)

Log-Ausgaben erfolgen auf der Konsole und in rotierende Log-Dateien (täglich, 14 Tage Aufbewahrung). Der Header `X-Correlation-Id` muss von jedem Service an ausgehende Requests angehängt und aus eingehenden Requests ausgelesen werden. Optional (Bonuspunkte): Anbindung eines zentralen Log-Stacks (ELK, Loki/Grafana).

3.4 Kommunikation zwischen Services

Folgende Kommunikationsmuster sind vorgegeben:

- * **Synchron (REST/HTTP):** Shop-Service zu Warehouse-Service, Shop-Service zu Billing-Service
- * **Asynchron (Message Queue oder Event-Bus):** Billing-Service zu Invoice-Service (PaymentSucceeded-Event), alle Services zu Audit-Service (Domain-Events)

Als Message Broker kann RabbitMQ, Kafka oder ein einfacher In-Process-Event-Bus verwendet werden. Die Wahl **muss** im Architekturdokument begründet werden.

3.5 API-Design

Alle REST-Schnittstellen müssen nach folgenden Regeln gestaltet werden:

- * Ressourcenorientierte URLs (kein RPC-Stil wie /createOrder)
- * Einheitliche Fehlerantworten nach RFC 7807 (Problem Details for HTTP APIs)
- * HTTP-Statuscodes semantisch korrekt (201 Created, 202 Accepted, 409 Conflict, 422 Unprocessable Entity, ...)
- * Alle Endpunkte mittels OpenAPI 3.0 ([swagger.yaml](#)) dokumentiert

4 Erweiterungsaufgaben (Bonus)

Die folgenden Aufgaben sind freiwillig und werden mit Bonuspunkten bewertet. Es wird empfohlen, zunächst alle Pflichtaufgaben abzuschließen.

4.1 Retry & Circuit Breaker (+5 Punkte)

Implementieren Sie einen **Circuit Breaker** (z.B. Resilience4j oder eigene Implementierung) für den Aufruf des Invoice-Service. Nach drei aufeinanderfolgenden Fehlern wechselt der Circuit Breaker in den Open-Zustand und lässt nach 30 Sekunden automatisch einen Test-Request zu (Half-Open). Zustandswechsel werden als Audit-Snapshot persistiert.

4.2 Idempotenz-Garantien (+5 Punkte)

Stellen Sie sicher, dass der Endpunkt `POST /orders` idempotent ist. Sendet ein Client denselben Request zweimal (identischer `Idempotency-Key`-Header), darf die Bestellung nur einmal ausgeführt werden. Der zweite Request muss dasselbe Ergebnis liefern, ohne die Daten erneut zu verändern.

4.3 Admin-Dashboard (+5 Punkte)

Entwickeln Sie ein einfaches Web-Frontend (beliebiges Framework) mit:

- * Übersicht aller Bestellungen mit aktuellem Status
- * Detailansicht: vollständige Audit-Timeline als Schritt-für-Schritt-Ablauf
- * Filterung nach Bestellstatus, Zeitraum und correlationId
- * Echtzeit-Aktualisierung per Server-Sent Events oder WebSocket

4.4 Asynchroner Webhook-Anbieter (+5 Punkte)

Erweitern Sie einen Anbieter-Stub, so dass er asynchrone Zahlungsbestätigung simuliert: `charge()` antwortet sofort mit status: PENDING. Nach konfigurierbarer Verzögerung sendet der Stub einen Webhook-Request an den Billing-Service, der dann Erfolg oder Misserfolg signalisiert. Der Billing-Service muss in diesem Modus den Saga-Ablauf korrekt weiterführen.

4.5 Zentrales Log-Management (+5 Punkte)

Binden Sie einen zentralen Log-Stack an (ELK oder Loki/Grafana). Alle Services senden strukturierte Logs zentral. Grafana/Kibana wird mit einem vorkonfigurierten Dashboard ausgeliefert, das mindestens zeigt:

- * Anzahl der Bestellungen pro Zeitintervall
- * Fehlerrate nach Service
- * Zeitreihe der Zahlungsversuche nach Ergebnis (Erfolg / Ablehnung / Timeout)

5 Technische Rahmenbedingungen

5.1 Freiheiten und Vorgaben

Aspekt	Vorgabe / Freiheit
Programmiersprache	Frei wählbar; Konsistenz innerhalb einer Gruppe wird empfohlen
Framework	Frei wählbar (Spring Boot, Express.js, FastAPI, ...)
Datenbank	Mind. eine relationale DB (PostgreSQL/MySQL/MariaDB/H2) für Audit-Snapshots
Deployment	Docker Compose ist Pflicht; alle Services per 'docker-compose up' startbar
Message Broker	RabbitMQ, Kafka oder In-Process-Event-Bus (Begründung im Architekturdokument)
API-Dokumentation	OpenAPI 3.0 (swagger.yaml) pro Service, Pflicht
Versionskontrolle	Git (GitHub/GitLab), strukturierte Commit-Historie, sinnvolle Branches
Externe Zahlungs-APIs	Keine echten API-Keys; ausschließlich Stubs/Mocks verwenden

5.2 Qualitätsanforderungen an den Code

- * Jeder Service hat eine eigene [README.md](#) mit Beschreibung, Voraussetzungen und Startanleitung.
- * Unit-Tests für die Payment-Fassade (mind. 5 Testfälle: Erfolg, Ablehnung, Timeout je Anbieter, Anbieterwechsel per Konfiguration).
- * Integrationstests für den Bestellprozess: Happy Path und mindestens zwei Fehlerszenarien.
- * Konfiguration ausschließlich über Umgebungsvariablen oder Konfigurationsdateien - keine hartcodierten Werte im Code.
- * Keine zirkulären Abhängigkeiten zwischen Services (Architekturverletzung wird bewertet).

6 Abgabumfang

6.1 Repository-Struktur (Mindestvorgabe)

Das GitHub-Repository muss folgende Verzeichnisstruktur aufweisen (begründete Abweichungen sind möglich):

```
/
|- docker-compose.yml
|- docs/
|   |- architecture.md           # Architekturdokumentation
```

```
|   |- decisions/                # Architecture Decision Records (ADRs)
|   +- diagrams/                # UML-Diagramme
|- shop-service/
|- billing-service/
|   +- src/.../payment/         # Payment-Fassade
|- invoice-service/
|- warehouse-service/
+- audit-service/
```

6.2 Architekturdokumentation

Das Dokument `docs/architecture.md` enthält mindestens:

- * **Systemkontext-Diagramm:** zeigt das System und seine externen Akteure (Kunde, Zahlungsanbieter-Stubs)
- * **Komponentendiagramm:** alle Services, ihre Schnittstellen und Kommunikationsbeziehungen
- * **Sequenzdiagramm Happy Path:** vollständiger Bestellablauf als UML-Sequenzdiagramm
- * **Sequenzdiagramm Fehlerszenario:** mind. ein Kompensationsablauf als UML-Sequenzdiagramm
- * **Architecture Decision Records (ADRs):** mind. 3 begründete Architekturentscheidungen (Wahl des Message Brokers, Persistenzstrategie für Audit-Snapshots, Kommunikationsmuster)
- * **Erweiterbarkeitsanalyse:** Wie wird ein vierter Zahlungsanbieter hinzugefügt? Welche Dateien werden geändert, welche nicht?

6.3 Schriftlicher Bericht

Zusätzlich zur technischen Dokumentation ist ein Bericht (PDF, 8-15 Seiten ohne Anhang) abzugeben mit folgenden Kapiteln:

1. Einleitung: Problemstellung, Ziele, Abgrenzung
2. Grundlagen: eingesetzte Architekturmuster (Fassade, Saga, Event Sourcing Light, ...)
3. Systemarchitektur: Beschreibung der Komponenten und ihrer Interaktionen
4. Implementierung: ausgewählte Detailbeschreibungen (Payment-Fassade, Audit-Snapshot-Schema, Kompensationslogik)
5. Qualitätssicherung: Teststrategie und Testergebnisse
6. Reflexion: Was lief gut? Was würden Sie anders machen? Welche Kompromisse wurden eingegangen?

6.4 Präsentation

Jede Gruppe präsentiert ihr Projekt in einem 20-minütigen Slot (15 min Vortrag + 5 min Fragen). Pflichtbestandteile:

- * Live-Demo: Bestellprozess Happy Path und mind. ein Fehlerszenario mit Saga-Kompensation
- * Demonstration des Audit-Logs für eine vollständige Bestellung (chronologische Snapshot-Kette)
- * Erläuterung der Payment-Fassade und wie ein Anbieterwechsel funktioniert
- * Kritische Reflexion: Was hätten Sie im Nachhinein anders entschieden?

7 Bewertungskriterien

Die Gesamtpunktzahl beträgt 100 Punkte (Pflicht) + maximal 20 Bonuspunkte.

Kriterium	Gewichtung	Beschreibung
Architektur & Design	25 Pkt.	Sinnvolle Servicegrenzen, korrekte Fassadenimplementierung, ADRs, Diagrammqualität
Bestellprozess & Saga	20 Pkt.	Vollständiger Happy Path, mind. 2 Fehlerszenarien mit korrekter Kompensation
Audit-Snapshot-System	20 Pkt.	Unveränderliche Snapshots, korrekte Verkettung, REST-Abfrage-Endpunkt
Logging-Qualität	10 Pkt.	Strukturiertes JSON-Logging, correlationId in allen Services, Log-Rotation
Code-Qualität & Tests	10 Pkt.	Testabdeckung Fassade, Integrationstests, Code-Organisation, keine Hardcoded-Werte
Dokumentation & Bericht	10 Pkt.	Vollständigkeit, Verständlichkeit, Diagrammqualität, ADRs
Präsentation & Demo	5 Pkt.	Verständlichkeit, Live-Demo-Erfolg, Antworten auf Fachfragen
Bonus: Circuit Breaker	+5 Pkt.	Korrekte Implementierung mit Audit-Snapshot-Integration
Bonus: Idempotenz	+5 Pkt.	Korrekte Idempotency-Key-Verarbeitung
Bonus: Admin-Dashboard	+5 Pkt.	Funktionsfähiges Frontend mit Audit-Timeline
Bonus: Log-Stack	+5 Pkt.	Vollständig integrierter Log-Stack mit vorkonfiguriertem Dashboard

Wichtiger Hinweis: Bitte stellen Sie sicher, dass die Anwendung in einer frischen Umgebung lauffähig ist.

8 Empfohlener Zeitplan

Tage	Phase	Ziel / Meilenstein
1-2	Planung & Design	Systemkontext, Komponentendiagramm, ADRs entwerfen; API-Kontrakte als OpenAPI-Dateien anlegen
3-5	Grundgerüst	Alle Services mit leerem Handler lauffähig; docker-compose.yml; Audit-Service Datenbankschema anlegen
6-8	Kernimplementierung	Bestellprozess Happy Path vollständig; Payment-Fassade mit beiden Stubs; Saga-Grundlogik
9-11	Fehlerszenarien & Audit	Alle Kompensationspfade implementiert; Audit-Snapshots in allen Services; Logging vollständig

12-13	Tests & Qualität	Unit-Tests Fassade; Integrationstests; API-Dokumentation vervollständigen; Code-Review
14	Bericht & Folien	Abschlussbericht schreiben; Präsentationsfolien vorbereiten
15	Puffer & Abgabe	Abschließende Tests in frischer Umgebung; Git-Repository aufräumen; Abgabe

9 Hinweise und häufige Fehler

9.1 Architektur

- * **Servicegrenzen zu fein:** Vermeiden Sie es, für jede Datenbankoperation einen eigenen Service zu erstellen. Services haben fachliche Verantwortungsbereiche, keine technischen.
- * **Direktzugriff auf fremde DBs:** Jeder Service hat seine eigene Datenbank. Kein Service greift direkt auf die Datenbank eines anderen Service zu.
- * **Synchrone Kaskaden:** Lange synchrone Aufrufketten erhöhen die Ausfallwahrscheinlichkeit. Nutzen Sie asynchrone Kommunikation wo möglich.

9.2 Fassade

- * **Fassade als reine Weiterleitung:** Die Fassade soll nicht nur Methoden delegieren, sondern auch Fehlerbehandlung, Logging und ggf. Retry kapseln.
- * **Anbieter-spezifische Typen durchreichen:** Die Fassade darf keine anbieter-spezifischen Typen nach außen geben. Alle Rückgabewerte sind eigene Domänentypen.

9.3 Audit & Logging

- * **correlationId fehlt:** Die correlationId muss vom ersten Request an bis zum letzten Service-Aufruf durchgereicht werden - als HTTP-Header `X-Correlation-Id`.
- * **Snapshot nach Update:** Snapshots dürfen nicht aktualisiert werden. Jeder Zustandsübergang erzeugt einen neuen Snapshot-Datensatz.
- * **Log-Level:** Technische Debug-Informationen auf DEBUG, Geschäftsereignisse auf INFO, erwartbare Fehler auf WARN, unerwartete Systemfehler auf ERROR.

10 Empfohlene Ressourcen

- * **Saga-Muster:** Richardson, C. - Microservices Patterns, Manning 2018, Kapitel 4
- * **Event Sourcing:** Fowler, M. - martinfowler.com/eaDev/EventSourcing.html
- * **Fassaden-Muster:** Gamma et al. - Design Patterns, Addison-Wesley, S. 185-193
- * **RFC 7807 (Problem Details):** tools.ietf.org/html/rfc7807
- * **Architecture Decision Records:** adr.github.io - Template nach Nygard
- * **OpenAPI 3.0:** spec.openapis.org/oas/v3.0.3
- * **Resilience4j:** resilience4j.readme.io (für Circuit Breaker Bonus)